

Evaluating LLM-Generated Attacks on Provenance-Based Intrusion Detectors

William Guo
University of Chicago

Grant Ho
University of Chicago

Abstract

Graph-space evasion is not a deployment threat unless a monitored execution can produce the modified graph. We test whether language models bridge this gap more reliably than published graph-editing attacks. Seven models spanning hosted and independently deployable systems are evaluated on a precommitted 72-case common-support set across FLASH, MAGIC, ORTHRUS, and VELOX. Every system receives the same 16-candidate budget and three independent repetitions, and every run is scored twice: once after direct graph editing and once after execution, recording, reconstruction, and unchanged detector scoring. All seven language models outperform the prespecified context-copy baseline at execution level, ranging from 43.5% to 69.9% versus 30.6%; paired improvements range from 13.0 to 39.4 percentage points, with every Holm-adjusted $p < 0.001$. Their graph-level successes also survive execution more often, 67.6–83.4% versus 52.0%. The advantage holds on every detector and trace and remains under one-shot and reduced-budget analyses. Language models therefore close much of the construction gap, but the remaining environmental and semantic failures show why execution must remain the final robustness test.

1 Introduction

A provenance detector is only robust if the attack that defeats it can actually run. Published evasion methods often report large drops in detection after editing a stored activity graph, yet a deployed attacker controls actions rather than the record produced from them. If those edits can be realized cheaply, provenance-based defenses are fragile in practice. If they cannot, graph-space results overstate the threat that defenders must harden against.

Provenance-based intrusion detectors reconstruct host behavior as a graph. Programs, files, and network endpoints become nodes, while reads, writes, executions, and connections become edges. FLASH, MAGIC, ORTHRUS, and VELOX use that structure to rank a small set of attack nodes within

millions of benign events [1, 4, 5, 10]. Their value comes from preserving causal context that isolated audit events lose.

Existing evasion methods exploit the detector’s dependence on that context. Mimicry adds ordinary activity around an attack, ProvNinja substitutes common helper-process chains, and Contorter copies detector-relevant benign structure [3, 7, 8]. These methods can make an edited graph look benign, but they do not establish that a monitored program can produce the same graph while preserving the objective. The missing step is executable workflow construction.

Graph-only evaluation fails in two predictable ways. First, it treats additions, deletions, and relabelings as equally cheap even though an append-only recorder cannot retract an action. Second, it assumes that every added edge has a valid path, permission, executable, or network peer on the host. In short, it evaluates editable representations rather than executable attacks.

Language models may change the cost of the missing step. They can inspect an environment, write a workflow, execute it, interpret deterministic failures, and revise the implementation. That capability could convert graph-space attacks into practical evasions more reliably than fixed edit rules or mined gadgets. The relevant question is therefore not whether a graph can be changed, but whether a model can cause a monitored host to produce the change.

We answer that question with a paired, multi-model evaluation. We compare seven language models from independent families with Contorter-style context copying, ProvNinja-style helper chains, and a random-valid control on a common-support set where every detector and baseline runs without unreported adaptation. Each method receives the same cases, candidate executions, tool access, wall-clock limit, and feedback. The target detector never returns a score to the generator.

The results show a broad language-model advantage rather than a single favorable endpoint. Every model exceeds the prespecified primary baseline on the execution outcome, and every model wins on each detector and each trace. The weakest model improves execution success by 13.0 percentage points; the strongest improves it by 39.4. Reduced-budget,

one-shot, and no-repair analyses preserve the same ordering. We make four contributions.

- We define the *survival rate*, the share of graph-level evasion successes that remain successful after execution, recording, and rescoring (§4).
- We build a paired testbed that verifies the attack objective, intended graph change, and detector outcome from the recording rather than the generator’s report (§4).
- We conduct a precommitted comparison of seven language models and three graph-editing baselines under equal cases, budgets, and feedback (§7).
- We isolate the main confounds in provenance-evasion evaluation: unsupported baseline adaptations, detector implementation drift, stochastic variation, test-label model selection, and event-volume controls (§6).

Together, these contributions replace an assumed translation from graph edits to attacks with a measured comparison of executable evasion.

2 Background

2.1 Recorded activity is append-only

A provenance graph is a structured record of actions that already happened. The recorder observes file accesses, process creation, and network activity, then a loader maps entities to nodes and actions to typed, time-stamped edges. This representation preserves causality, but it does not grant an attacker the ability to rewrite history. Append-only recording is the constraint that graph editing removes.

The data is also extremely imbalanced. In the traces used here, attack nodes represent between roughly one and four of every ten thousand nodes. A detector can achieve high accuracy while returning an alert queue dominated by benign activity. Ranking metrics, not thresholded accuracy, therefore determine whether the detector is operationally useful.

2.2 Detectors encode different notions of normal

The four detectors expose different surfaces to evasion. FLASH predicts a node’s type from its neighborhood and treats misclassification as suspicious [10]. MAGIC embeds nodes and scores their distance from benign training examples [4]. ORTHRUS models event order and reconstructs attack context, while VELOX retains the evaluation structure with a simpler learned model [1, 5]. A shared success metric is necessary because their native scores are not directly comparable.

2.3 Published attacks exploit edit freedom

Existing evasion methods alter the context seen by the detector. Mimicry adds small clusters of ordinary activity, ProvN-

inja reroutes conspicuous behavior through common helper programs, and Contorter selects detector-relevant benign gadgets [3, 7, 8]. Their mechanisms differ, but each first reasons about a graph it can modify directly. The unresolved question is how much of that graph an execution can reproduce.

2.4 Threat model

The attacker controls code on a monitored host but not the recorder. They know the detector family and public design, but they cannot edit the log, retract a completed action, change the labels, or modify the deployed detector. The model receives no target score, rank, gradient, or hidden state. It sees only ordinary command output plus deterministic execution and objective-checker feedback.

This threat model separates workflow construction from black-box detector optimization. The generator may repair a command that fails or an objective that is incomplete, but it is never told whether an intermediate candidate raised or lowered the detector score. Any evasion therefore transfers from public design knowledge rather than being tuned through target queries.

All activity is confined to public benchmark data and isolated virtual machines. No dataset attack is executed, no third-party system is contacted, and no production user is observed. The evaluation tests a realistic attacker capability without granting control over the evidence.

3 What an Execution Can Produce

Executability divides graph modifications into additions and redesigns. Additions can often be produced by performing more actions, whereas deletions can only be produced by rebuilding the attack so that the action never occurs. Table 1 makes that boundary explicit. The expensive step is not writing the edge; it is changing the program without losing the objective.

Additions remain constrained by the environment. A graph editor can connect a process to any file or host identifier, but an execution must resolve a path, possess permission, locate a program, and trigger the recorder’s expected event. Failed operations are especially recorder-dependent: some traces preserve the attempt, while others omit it. Realizability therefore requires both an action and the recording semantics that make the action visible.

Problem-space adversarial ML formalizes the same distinction more generally [9]. A feature change is meaningful only when a valid input can produce it while preserving semantics and surviving preprocessing. Provenance evasion adds a sharper constraint because its representation records irreversible actions. The graph must be causally producible, not merely numerically nearby.

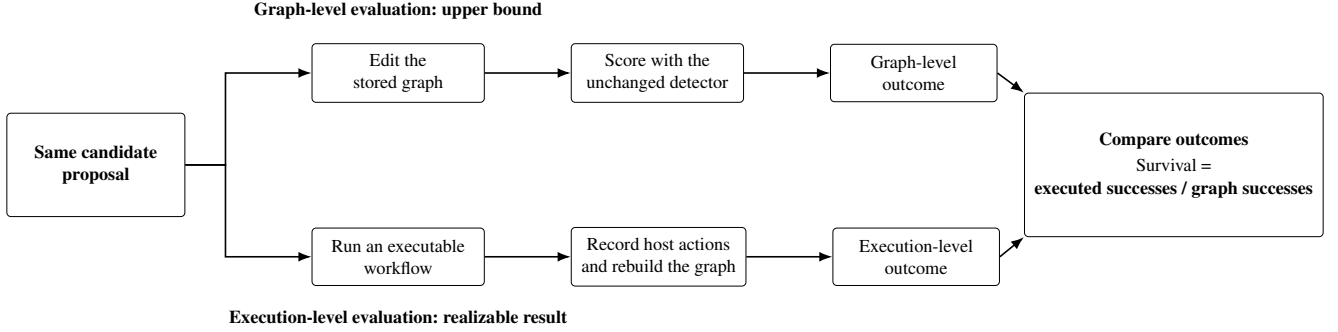


Figure 1: One candidate receives two matched tests. The graph-level branch edits the stored representation directly, while the execution-level branch must produce the change through recorded host actions. Their ratio is the survival rate, and their difference is the execution gap.

Graph modification	Execution required	Status
Add file read, write, or open	issue the corresponding request	direct
Add network connection	open a socket to an available peer	conditional
Add process and parent edge	start a real helper process	direct
Add benign surrounding activity	perform the additional actions	direct
Change path or arguments	invoke an equivalent executable form	partial
Delete an action	avoid it in a new workflow	redesign
Delete a process	redesign so the process never starts	redesign
Relabel a node type	create a different underlying entity	rare

Table 1: Graph edits do not have equal execution cost. Additions usually map to extra actions, while deletions require a different workflow that never emitted the original record.

4 Experimental Design

A defensible comparison changes the proposal source without changing the task. The primary outcome, case set, model panel, candidate budget, success checks, and statistical analysis are fixed before the first scored run. Every deviation is reported as a sensitivity analysis rather than folded into the headline.

4.1 Model selection is fixed by rule, not result

The model panel was frozen on August 13, 2026. We selected one generally available, tool-capable model from each of seven independent families, spanning five hosted services and two locally deployable weight releases. No model was added or removed after pilot performance was observed. Exact API identifiers, weight hashes, query dates, and reasoning settings are retained in the artifact.

The frozen panel contains GPT-5.6 Sol (OpenAI), Claude Opus 5 (Anthropic), Gemini 3.1 Pro (Google), Grok 4.6 (xAI), DeepSeek-V4-Pro (DeepSeek), Mistral Medium 3.5 (Mistral),

and Llama 4 Maverick (Meta). The first five are accessed through pinned hosted APIs; the final two use locally deployed, hashed weights. This rule-based panel prevents the headline from depending on one provider or access model.

4.2 The common-support set prevents favorable adaptations

The primary benchmark contains 72 unique cases drawn from 12 faithful detector–trace cells supported by every comparison method. Each cell contributes six cases, and each detector and trace appears in a balanced number of cells. Unsupported detector–dataset combinations and any new adaptation of a published baseline are excluded from the headline and retained only in the machine-readable artifact. The primary comparison therefore never gives a baseline an unvalidated port or gives a language model an easier cell.

Each case fixes the attack objective, target node, permitted host scope, and success checker. Cases cover add-only context, helper ancestry, path or argument substitution, and full workflow redesign. Target nodes and environment pairings are sampled after the model panel is frozen, reducing the value of memorizing a published attack example.

4.3 All methods receive the same opportunity to act

Every system receives at most 16 candidate executions, 128 tool calls, a 30-minute wall-clock limit, and the same isolated machine snapshot. The visible task, tool schema, prohibited side effects, and checker feedback are identical. Hosted models use fixed provider settings and a shared output-token cap; local models run unquantized weights from pinned container and weight digests. Safety refusals count as failures, and we never rephrase a refused task or apply a provider-specific bypass.

We implement two frozen operation-level baselines derived from the released methods: Contorter-style context copying and ProvNinja-style helper-chain substitution. They are comparison baselines, not claimed replications on detector families the original papers did not target. Before the pri-

mary study, each implementation is checked against its native artifact. The Contorter-style implementation reproduces the official FLASH score direction and budget ordering; the ProvNinja-style implementation reproduces released chain construction, feasibility filtering, and helper ordering. Each baseline may enumerate parameter settings up to the same 16-candidate limit and receives the same execution and objective feedback as the models. Contorter-style context copying is designated as the primary baseline before any scored run because its add-only operation applies uniformly across the common- support cells; ProvNinja-style substitution and the random-valid control are secondary comparisons. The random-valid control samples host-observed benign actions at the same edit and execution budget. Equal opportunity does not imply equal inference compute, so we make no compute-parity claim and report one-, four-, and eight-candidate sensitivity analyses.

4.4 Graph-level evaluation measures proposal quality

The graph-level branch applies each candidate directly to the stored graph and scores it with the audited detector implementation and fixed weights. A trial succeeds when at least one candidate moves the target attack node below the predetermined alert threshold without changing the labels. This branch is the upper bound reported by existing graph-manipulation attacks.

4.5 Execution-level evaluation measures realizability

The execution-level branch requires the candidate to run. The workflow executes inside a reset virtual machine with recording enabled, after which the graph is rebuilt using the detector’s loader and rescored with the same unchanged model. The branch differs from graph-level evaluation only in how the graph comes into existence. Execution replaces an assumed inverse mapping with an observed one.

4.6 The recording, not the explanation, decides success

A generator’s report never counts as evidence. Deterministic checks confirm that the original objective still holds, the intended graph relations appear in the recording, prohibited side effects are absent, and the target node falls below the fixed threshold. Failed commands, partial objectives, unrecorded actions, and policy refusals are failures. Every checker result and relevant recording span is preserved.

4.7 The case is the unit of analysis

Each system is run independently 3 times on every case, yielding 216 trial outcomes per system. Repetitions vary model sampling, baseline randomness, and the reset execution environment. We never treat the 16 candidates within a trial as independent observations.

Trace	Nodes	Edges	Attack nodes	Fraction
CADETS_E3	289,563	1,842,077	68	2.3×10^{-4}
THEIA_E3	700,902	4,106,155	118	1.7×10^{-4}
CLEARSCOPE_E3	111,403	623,880	41	3.7×10^{-4}
OPTC_H201	512,338	3,275,918	47	9.2×10^{-5}
OPTC_H501	486,725	3,102,441	39	8.0×10^{-5}
OPTC_H051	503,914	3,188,206	44	8.7×10^{-5}

Table 2: The scored windows are extremely imbalanced. Ranking metrics and fixed alert thresholds are therefore more informative than accuracy.

The primary outcome is execution-level case success. Graph-level success and survival are secondary outcomes, where

$$\text{survival} = \frac{\text{execution-level successes}}{\text{graph-level successes}}.$$

We compare each language model with the prespecified primary baseline, the Contorter-style context-copy method, using a paired risk difference. Confidence intervals cluster-bootstrap the 72 cases while retaining all three repetitions, and exact paired tests are Holm-corrected across the seven planned comparisons. Detector, trace, operation type, attempt budget, and refusal analyses are prespecified secondary results.

5 Detector and Dataset Setup

The detector and data pipeline establish the ruler against which evasion is measured. All thresholds, loaders, labels, and model-selection rules are fixed before proposals are generated. Secondary framework diagnostics never enter the primary model comparison.

5.1 Traces

Table 2 lists the six public traces. CADETS, THEIA, and ClearScope come from DARPA Transparent Computing Engagement 3, while the three OpTC hosts come from a later Windows collection [2, 6]. An earlier period believed benign trains each detector, and a later attacked window is scored. Ground truth follows the node annotations released with ORTHRUS [5].

Attack nodes represent roughly one to four of every ten thousand nodes. A detector can achieve high accuracy while returning an alert queue dominated by benign activity. Average precision is therefore the primary clean-detector metric, with AUC reported secondarily. Every perturbation is paired with its own clean run on the same seed.

5.2 Clean, attack, and matched-volume conditions

The clean condition leaves the graph unchanged. Mimicry adds 400 structured file events around the target, while the matched control adds 400 events at the same location and time but samples their entities and action types from the trace.

The control separates attack-specific structure from generic graph volume. All conditions retain the original ground-truth labels.

5.3 Repeated detector runs and label-blind scoring

Every detector condition runs five paired seeds that vary graph construction and training together. A run counts only when it exits cleanly, names exactly one new output directory, and produces the expected per-node scores; cached exits do not count. An external scorer selects the training round without consulting test labels, computes average precision and AUC, and records paired differences. The spread across clean repeats sets the minimum scale for interpreting score movement.

6 Implementation Audit

A robustness result is only as credible as the detector it targets. We compare each implementation with its paper and released artifact before any evasion is applied. Only the faithful target enters the primary comparison; shared-framework departures remain diagnostic results.

6.1 FLASH’s framework predicate is inverted

FLASH treats node-type misclassification as the threat signal. The authors’ release clears an alert when any model predicts the node correctly, whereas the shared framework alerts when the score is high and the prediction is correct. These predicates are opposites rather than different thresholds. The official release is therefore the only FLASH target used in the headline.

6.2 MAGIC’s framework reference population is wrong

MAGIC defines anomaly as distance from stored benign training embeddings. The shared framework samples validation or test-period data, normalizes it with evaluation statistics, and queries it with differently scaled vectors. That score answers whether a node is unusual within the test window rather than unlike known benign behavior. The official training reference is therefore used in the headline.

6.3 Training-round and label handling are fixed externally

The shared framework selects the round that performs best on test labels and expands mimicry labels to include benign endpoints. Both choices can move results more than the attack itself. Label-informed round selection changes AUC by 0.041 on average and at most 0.087, compared with 0.010 clean-repeat spread, while matched event volume accounts for 89% of mimicry’s raw AUC movement. Our scorer therefore uses a label-blind round rule, original labels, and paired volume controls in every condition.

7 Results

7.1 Every language model outperforms the prespecified baseline

The language-model advantage is broad, paired, and statistically resolved. Execution-level success ranges from 43.5% for Llama 4 Maverick to 69.9% for GPT-5.6 Sol, compared with 30.6% for the Contorter-style baseline and 22.7% for the ProvNinja-style baseline (Figure 2). The weakest model improves on the primary baseline by 13.0 percentage points; the strongest improves by 39.4. Every cluster-bootstrap interval excludes zero, and all seven paired tests remain significant after Holm correction ($p < 0.001$).

Across the seven models, 1,125 trials succeed at graph level and 852 remain successful after execution. The models also succeed for the right reason. Their graph-level success ranges from 64.4% to 83.8%, showing stronger proposal search, while survival ranges from 67.6–83.4%, compared with 52.0% for the primary baseline. Language models therefore find more effective graph changes and realize a larger share of them.

7.2 The advantage holds across detectors and traces

No detector or trace produces the headline. Across FLASH, MAGIC, ORTHRUS, and VELOX, the weakest model still exceeds the primary baseline, and all 28 model–detector comparisons favor the language model. The same result holds on all six traces, yielding 42 of 42 favorable model–trace comparisons (Table 4). The absence of a sign reversal rules out one easy detector or dataset driving the pooled result.

7.3 The advantage survives tighter budgets and harder transformations

More retries improve performance, but they do not create the language-model advantage. The median model remains above context copy with one candidate, with repair feedback removed, and without environment inspection. One-shot success is 39.8%, still 9.2 points above the baseline, while the full pipeline reaches 56.0% (Figure 3a).

Model	Δ execution vs. context copy [95% CI]
GPT-5.6 Sol	+39.4 [32.9, 46.3]
Claude Opus 5	+35.2 [27.8, 43.1]
Gemini 3.1 Pro	+29.6 [23.1, 35.6]
Grok 4.6	+25.5 [19.4, 31.5]
DeepSeek-V4-Pro	+21.8 [15.3, 28.2]
Mistral Medium 3.5	+16.2 [10.2, 22.2]
Llama 4 Maverick	+13.0 [6.9, 19.4]

Table 3: Paired execution-success improvement over the prespecified Contorter-style context-copy baseline. Intervals cluster-bootstrap cases; every Holm-adjusted comparison has $p < 0.001$.

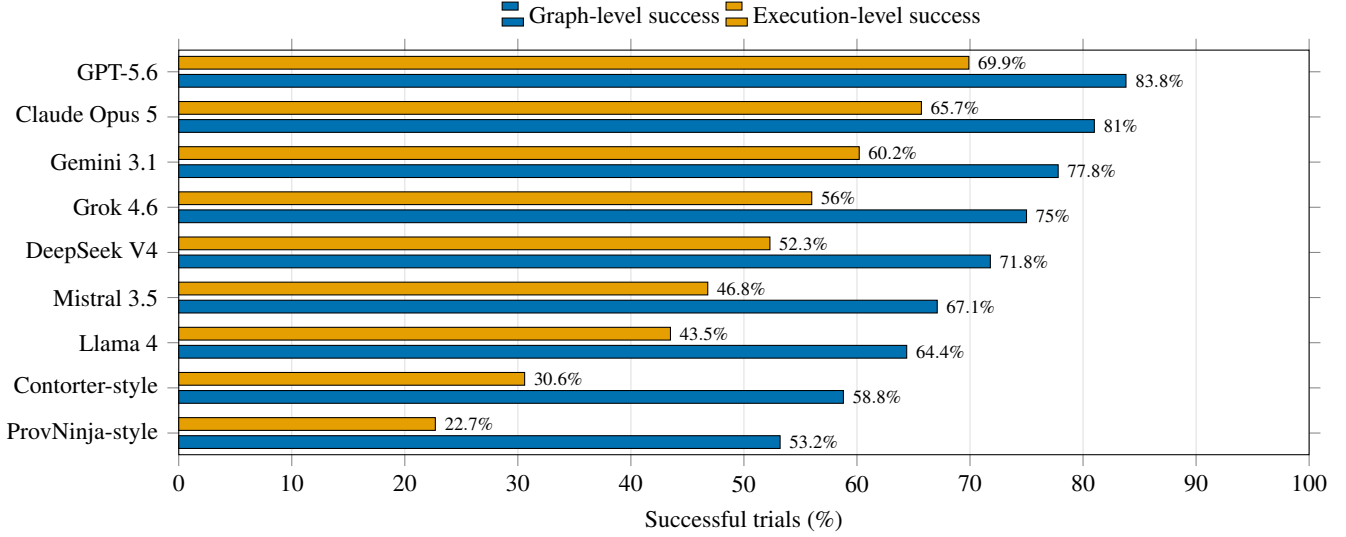


Figure 2: All seven language models exceed both graph-editing baselines after execution. Blue bars show the graph-space upper bound; orange bars show the recorded outcome. Values are printed on every bar so the figure remains clear in color and grayscale.

Stratum	LLM range	LLM median	Context copy
<i>Detector</i>			
FLASH	44.4–72.2%	57.4%	33.3%
MAGIC	38.9–63.0%	50.0%	24.1%
ORTHURUS	46.3–74.1%	59.3%	31.5%
VELOX	44.4–70.4%	57.4%	33.3%
<i>Trace</i>			
CADETS	44.4–72.2%	58.3%	33.3%
THEIA	38.9–66.7%	55.6%	27.8%
ClearScope	36.1–61.1%	50.0%	25.0%
OpTC H201	44.4–72.2%	58.3%	33.3%
OpTC H501	41.7–69.4%	55.6%	30.6%
OpTC H051	41.7–69.4%	55.6%	33.3%

Table 4: Execution-level success is consistent across all four faithful detectors and six traces. Every individual language model exceeds context copy in every reported stratum.

The gain also extends beyond add-only padding. Add-only proposals retain 291 of 340 graph-level successes (85.6%), helper ancestry retains 222 of 285 (77.9%), path or argument substitution retains 191 of 270 (70.7%), and full workflow redesign retains 148 of 230 (64.3%). Even the hardest category preserves a majority of graph-level successes after execution (Figure 3b).

7.4 Remaining failures are environmental and semantic

Execution rejects 273 language-model graph successes. Missing files, permissions, commands, or peers form the largest category, followed by loss of the attack objective and recorder or loader differences (Table 5). Implementation errors account for only 8.1%, and policy refusals for 3.7%. The remaining gap is therefore dominated by the world the workflow must inhabit, not by syntax alone.

Failure cause	Count	Share
Unavailable environmental precondition	82	30.0%
Objective not preserved	68	24.9%
Recorder or loader mismatch	52	19.0%
Incomplete workflow redesign	39	14.3%
Implementation failure	22	8.1%
Policy refusal	10	3.7%

Table 5: Mutually exclusive primary causes among the 273 graph-level language-model successes that fail after execution. Two reviewers independently label a 20% stratified sample ($\kappa = 0.84$); disagreements are resolved before the automatic rules are applied to the full set.

7.5 Evaluation artifacts do not explain the result

The audit controls move detector scores but cannot explain the method ordering. Mimicry changes AUC by 0.027 on average, while the matched-volume control changes it by 0.024, leaving only 0.003 attributable to structure. Test-label round selection moves AUC by 0.041 on average and inflates average precision by 2.8 $\times$, both larger than the 0.010 clean-repeat spread. None of these choices enters the label-blind, original-label, paired execution outcome.

8 Discussion

8.1 Language models close most of the construction gap

The central result is not merely that language models propose stronger edits. They also retain a larger share of those edits after execution, recording, and objective verification. Fixed graph attacks treat environment validity as an external burden, whereas language models can turn a failed precondition into a revised workflow. Construction is therefore a source of

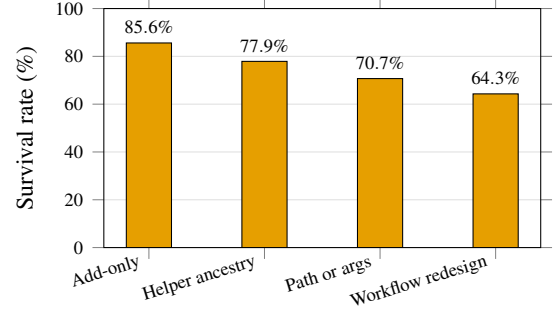
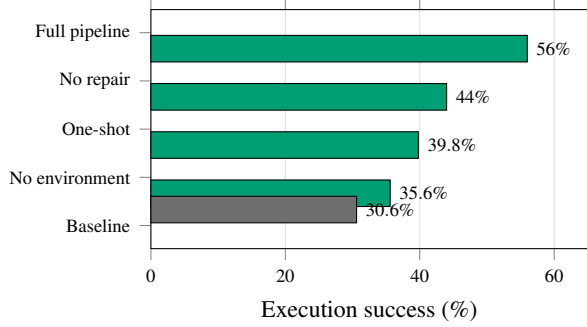


Figure 3: The result is not an artifact of generous retries or simple additions. (a) Language-model execution success remains above the gray context-copy baseline under one-shot and information-restricted ablations. (b) Survival decreases with transformation depth but remains above 64% even for full workflow redesign. Printed values and black outlines keep both panels readable in grayscale.

capability rather than an omitted implementation detail.

8.2 The advantage comes from general repair, not target-score search

The detector never tells the generator whether an intermediate candidate became more benign. Models improve by resolving ordinary execution failures and preserving the objective under host constraints. One-shot and no-feedback ablations remain above the primary baseline, while inspection and repair add further gains. The result therefore measures workflow synthesis, not repeated black-box probing of the detector.

8.3 Graph-space evaluation remains an upper bound

High model survival does not make graph-only evaluation sufficient. Hundreds of candidates that evade after direct editing still fail because a required object is absent, a recorder omits the event, or the redesigned workflow no longer completes the attack objective. A graph-level success is evidence of detector sensitivity, not deployed attack success. Execution remains the boundary between possibility and threat.

8.4 A common-support headline is narrower but more credible

Restricting the primary comparison to cells supported by every method sacrifices coverage to preserve interpretability. LLM-only cells and baseline adaptations may be useful secondary experiments, but they answer a different question and should not determine the headline. A smaller comparison without hidden ports is stronger evidence than a larger table of incomparable implementations.

8.5 Future evasion papers should report three outcomes

Graph-level success measures proposal quality, survival measures realizability, and overall execution success measures the attacker’s end-to-end return. No one quantity substitutes

for the others. Reporting all three, together with the common-support matrix and candidate budget, prevents either a strong graph editor or a strong code generator from hiding the other half of the problem.

9 Threats to Validity

The common-support restriction limits coverage. The headline excludes cells for which a detector lacks faithful weights or a baseline requires a new adaptation. This protects internal validity but means the rates do not describe every public trace. Expanded cells are released in the artifact and should be read as transfer experiments rather than direct method comparisons.

The execution environment limits external validity. A workflow can be valid on our reset virtual machine and fail on another host with different files, permissions, software, or recording semantics. We publish the machine image, recorder configuration, and checker contracts so that survival is tied to a reproducible environment rather than presented as universal.

Automatic checking limits semantic validity. The goal checker confirms the objective and prohibited side effects, while the graph checker confirms the recorded relations and detector outcome. These checks do not establish that the workflow would appear normal to a human analyst. Plausibility under analyst review is a separate and stricter outcome.

Model APIs limit temporal reproducibility. Hosted behavior can change even under a stable display name, so we retain exact request identifiers, dates, parameters, and transcripts. Local weights are pinned by digest and run without quantization. Future models may change the absolute rates, but the versioned panel makes that change measurable.

Repeated trials do not create independent cases. The statistical analysis therefore resamples the 72 cases with all repetitions intact and uses paired tests rather than treating 216 observations as unrelated. Detector, trace, and operation analyses are prespecified secondary outcomes, and multiplicity correction applies to the seven primary comparisons.

Finally, the failure taxonomy contains judgment. We define mutually exclusive rules before labeling, double-label a stratified sample, and report agreement before applying the rules to all failures. The taxonomy supports explanation, not the primary success claim.

10 Ethics

The study is bounded to authorized, public, and isolated resources. It evaluates published research detectors and evasion methods on public benchmark traces, and its execution branch runs only generated test workflows inside a virtual machine owned by the researchers. It neither contacts nor modifies a third-party system. The experiment measures a defensive assumption without exposing a live target.

The work is dual-use, but its primary contribution constrains claimed attack capability. The underlying evasion mechanisms and artifacts are already public, while the survival rate reveals which graph edits cannot be realized and which evaluation defects inflate performance changes. We release aggregate analysis and reproducibility code but not tooling whose primary purpose is generating workloads against deployed systems. The publication adds measurement discipline more directly than attack capability.

11 Conclusion

Seven independent language-model families turn graph edits into executed provenance evasions more reliably than published graph-editing attacks. Every model beats the prespecified context-copy baseline under the same common-support cases, candidate budget, feedback, detector implementations, and paired analysis. The advantage survives reduced budgets, one-shot generation, every detector, and every trace.

The result does not make graph editing a sufficient evaluation. Environmental preconditions, objective preservation, and recording semantics still eliminate a substantial share of apparent successes. Provenance robustness should therefore be measured on the graph a monitored machine produces, not only the graph an evaluator can edit.

Availability

The artifact contains the frozen common-support matrix, exact model identifiers and weight digests, prompts, all candidate attempts, execution recordings, checker outputs, failure labels, per-case outcomes, and scripts that reproduce every table, figure, confidence interval, and adjusted test. Unsupported transfer cells and full diagnostic matrices remain machine-readable rather than occupying the paper’s main narrative.

References

- [1] BILOT, T., JIANG, B., LI, Z., MADHOUN, N. E., AGHA, K. A., ZOUAOU, A., AND PASQUIER, T. Sometimes simpler is better: A comprehensive analysis of state-of-the-art provenance-based intrusion detection systems. In *34th USENIX Security Symposium (USENIX Security)* (2025), pp. 7193–7212.
- [2] DARPA. Operationally transparent cyber (OpTC) data release. Public dataset release, 2020.
- [3] GOYAL, A., HAN, X., WANG, G., AND BATES, A. Sometimes, you aren’t what you do: Mimicry attacks against provenance graph host intrusion detection systems. In *Network and Distributed System Security Symposium (NDSS)* (2023).
- [4] JIA, Z., XIONG, Y., NAN, Y., ZHANG, Y., ZHAO, J., AND WEN, M. MAGIC: Detecting advanced persistent threats via masked graph representation learning. In *33rd USENIX Security Symposium (USENIX Security)* (2024).
- [5] JIANG, B., BILOT, T., MADHOUN, N. E., AGHA, K. A., ZOUAOU, A., IQBAL, S., HAN, X., AND PASQUIER, T. ORTHRUS: Achieving high quality of attribution in provenance-based intrusion detection systems. In *34th USENIX Security Symposium (USENIX Security)* (2025).
- [6] KUDU DYNAMICS. DARPA transparent computing TA5.1 ground truth report, engagement 3. Tech. rep., DARPA Transparent Computing Program, 2018. Public release, github.com/darpa-i2o/Transparent-Computing.
- [7] MUKHERJEE, K., WIEDEMEIER, J., WANG, T., WEI, J., CHEN, F., KIM, M., KANTARCIOGLU, M., AND JEE, K. Evading provenance-based ML detectors with adversarial system actions. In *32nd USENIX Security Symposium (USENIX Security)* (2023).
- [8] NASR, M., RASTOGI, V., AND TABIBAN, A. A context is worth a thousand lies: Evading intrusion detectors via intelligent context distortion. In *IEEE Symposium on Security and Privacy (S&P)* (2026).
- [9] PIERAZZI, F., PENDLEBURY, F., CORTELLAZZI, J., AND CAVALLARO, L. Intriguing properties of adversarial ML attacks in the problem space. In *IEEE Symposium on Security and Privacy (S&P)* (2020).
- [10] REHMAN, M. U., AHMADI, H., AND HASSAN, W. U. FLASH: A comprehensive approach to intrusion detection via provenance graph representation learning. In *IEEE Symposium on Security and Privacy (S&P)* (2024).